

T09 — Array, Matrici e Tipi Enumerativi (enum)

Array Mono e Multidimensionali, Enum Tipizzati, Immutabilità e Singleton in SimpleDate v4

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

In questo blocco approfondiamo la gestione a basso livello degli array nello Heap, i tipi enumerati (enum) ([T09 - Arrays and Enumerative Types.pdf](#)) e analizziamo il codice di [Lezione09](#), dove Date diventa una **classe immutabile** (campi final e rimozione dei setter) e compare per la prima volta un design pattern fondamentale: il **Singleton Pattern** implementato in [BirthDay.java](#).

0.0.1 1. Teoria: Concetti Teorici dalle Slide (Slide T09)

A. Gli Array in Java (Slide 1–13)

- **Natura degli Array:**

- Un array è una sequenza contigua e ordinata di variabili omogenee (dello stesso tipo), accessibili tramite indice numerico intero che parte da 0.
- In Java gli array **non sono tipi primitivi**: sono **oggetti speciali allocati nello Heap**.
- Possono memorizzare valori primitivi oppure **riferimenti/puntatori a oggetti**, mai oggetti per valore diretto.
- La dimensione di un array può essere stabilita dinamicamente a runtime all'atto dell'allocazione (`new int[size]`), ma è **immutabile una volta creata**.

- **Dichiarazione vs Creazione:**

- Sintassi raccomandata: `int[] arr;` (la notazione `int arr[]`; è ammessa per retrocompatibilità col C, ma sconsigliata).
- La dichiarazione `int[] arr;` alloca solo una variabile reference nello Stack (inizializzata a `null`), **senza riservare spazio per gli elementi nello Heap**.
- Allocazione:
 - * Tramite `new`: `float[] arr = new float[10];` (gli elementi assumono il valore di default del tipo, es. `0.0f`).
 - * Tramite inizializzazione statica: `int[] primes = {2, 3, 5, 7};` (la dimensione viene inferita automaticamente dal compilatore).

- **Il Pseudo-Campo `.length`:**

- Ogni array possiede la proprietà `public final int length`.

- Attenzione: **Distinzione d'esame**: per gli array si scrive `arr.length` (**senza parentesi**, è una variabile *final*), mentre per le stringhe si invoca il metodo `str.length()` (**con le parentesi**).
- Se si tenta di leggere `arr.length` su un array nullo (`arr = null;`), la JVM lancia una `NullPointerException`.

- **Confronto e Stampa degli Array (I Trabocchetti Classici):**

1. `arr1 == arr2`: confronta solo gli indirizzi nello Heap. Se due array contengono gli stessi identici elementi ma risiedono in aree diverse, restituisce `false`.
2. `arr1.equals(arr2)`: **NON funziona**. Gli array non sovrascrivono il metodo `.equals()` di `Object`, quindi `equals` si comporta esattamente come `==`.
3. **Soluzione standard**: per confrontare il contenuto si usa il metodo statico `java.util.Arrays.equals(arr1, arr2)`.
4. `arr.toString()`: stampa la firma bytecode della reference (es. `[I@6ce253f1`, dove `[` indica un array e `I` indica il tipo `int`).
5. Per visualizzare gli elementi a video in formato leggibile: `java.util.Arrays.toString(arr)`.

- **Enhanced For Loop (For-Each, Java 5+):**

```
1 for (String arg : args) {
2     System.out.println(arg);
3 }
```

Sostituisce il `for` contatore eliminando il rischio di errori di *off-by-one* e `ArrayIndexOutOfBoundsException`.

- **Array Multidimensionali e Matrici Frastagliate (Jagged Arrays):**

- In Java non esistono matrici bidimensionali a blocco contiguo in stile Fortran/C: **un array multidimensionale è un array di riferimenti ad altri array**.
- *Conseguenze pratiche*:
 1. **Scambio di righe in tempo $O(1)$** : per invertire la prima e l'ultima riga di una matrice non serve copiare gli elementi uno per uno, basta scambiare i loro puntatori:

```
1 int[] temp = matrix[0];
2 matrix[0] = matrix[matrix.length - 1];
3 matrix[matrix.length - 1] = temp;
```

2. **Righe a lunghezza eterogenea (Jagged/Pyramid Arrays):**

```
1 int[][] pyramid = new int[3][];
2 pyramid[0] = new int[1]; // Riga 0 ha 1 elemento
3 pyramid[1] = new int[2]; // Riga 1 ha 2 elementi
4 pyramid[2] = new int[3]; // Riga 2 ha 3 elementi
```

3. Per stampare matrici annidate, `Arrays.toString(matrix)` stampa gli indirizzi delle singole righe. Si deve iterare sulle righe o usare `java.util.Arrays.deepToString(matrix)`.

B. Tipi Enumerati (enum) (Slide 14–19)

- Introdotti in Java 5 con la keyword `enum` per gestire insiemi chiusi e prefissati di valori costanti (es. giorni della settimana, punti cardinali, lingue).
- **Regole e Caratteristiche**:
 - Possono essere dichiarati in un file `.java` dedicato (come classe autonoma) o all'interno di una classe come membro statico, ma **mai all'interno di un metodo**.

- Non sono semplici interi o stringhe: sono classi speciali che estendono implicitamente `java.lang.Enum`. Il loro costruttore è privato e non può essere invocato con `new`.

- **Metodi Predefiniti Fondamentali:**

- * `e.name()`: restituisce il nome letterale della costante come `String` (es. "WEST").
- * `e.ordinal()`: restituisce la posizione ordinale intera partendo da 0 (es. 3).
- * `EnumClass.values()`: restituisce un array contenente tutte le costanti dichiarate nell'enum, comodo per cicli `for-each`.

- **Confronto tra Enum: Perché usare `==` invece di `.equals()`:**

- La JVM garantisce che in memoria esista **una e una sola istanza** per ciascuna costante di un enum (*garanzia di unicità singleton*).
- Pertanto, il confronto con `==` è perfettamente sicuro ed è **più robusto di `.equals()`**: se una variabile vale `null`, l'espressione `move == Direction.WEST` restituisce `false` senza errori, mentre `move.equals(...)` causerebbe un crash con `NullPointerException`!

- **Uso di Enum negli `switch`:**

- All'interno dei blocchi `case`, **non si qualifica** il tipo: si scrive `case IT:` e non `case Language.IT:.`

0.0.2 2. Codice: Evoluzione del Codice: `SimpleDate` (Lezione 09)

In [Lezione09](#), il docente introduce due cambiamenti strutturali fondamentali:

1. Immutabilità di `Date.java`

```

1 public class Date {
2     - private int day;
3     - private int month;
4     - private int year;
5     + private final int day;
6     + private final int month;
7     + private final int year;
8
9     // Costruttori...
10
11     - public void setDay(int day) { ... }
12     - public void setMonth(int month) { ... }
13     - public void setYear(int year) { ... }

```

- **Campi `final`:** Una volta inizializzati nel costruttore, `day`, `month` e `year` non possono più essere riassegnati.
- **Rimozione dei Setter:** I metodi mutatori `setDay`, `setMonth` e `setYear` vengono eliminati.
- **Pattern Immutabile:** Un oggetto `Date` è ora a prova di manomissione. Se un programma vuole calcolare "il giorno successivo", non può mutare l'oggetto esistente, ma **deve istanziare un nuovo oggetto `Date`** con le coordinate aggiornate (esattamente come avviene per le `String`).

2. Il Design Pattern Singleton: `BirthDay.java`

```

1 public class BirthDay {
2     private static Date date = new Date(1, 1, 1970);
3     private static BirthDay instance; // Riferimento all'unica istanza
4
5     // 1. Costruttore privato: impedisce istanziazioni esterne con 'new BirthDay()'
6     private BirthDay() { }
7

```

```

8 // 2. Metodo statico di fabbrica con Lazy Initialization
9 public static BirthDay getInstance() {
10     if (instance == null)
11         instance = new BirthDay();
12     return instance;
13 }
14
15 public Date getDate() {
16     return date;
17 }
18 }

```

Analisi: Analisi Architetture del Singleton:

- **Scopo del Pattern:** Garantire che per tutta la durata dell'applicazione esista **una e una sola istanza** di una classe nello Heap e fornire un punto di accesso globale ad essa.
- **I Tre Pilastri del Singleton in Java:**
 1. **Costruttore private:** blocca l'accesso a `new BirthDay()` da qualsiasi altra classe (tentare di chiamarlo genera un errore a compile-time).
 2. **Campo statico privato (instance):** contiene l'unico riferimento all'oggetto.
 3. **Metodo factory pubblico statico (getInstance()):** implementa la *Lazy Initialization* (crea l'oggetto solo alla prima richiesta, riusandolo per tutte le successive).

3. Verifica in `MainDate.java` e il Bug dei Getter~

```

1 BirthDay bDay = BirthDay.getInstance();
2 // BirthDay day1 = new BirthDay(); // COMPILER-TIME ERROR!
3 BirthDay day1 = BirthDay.getInstance();
4 System.out.println("bDay ?= day1: " + (bDay == day1)); // STAMPA TRUE!
5
6 Date today = new Date(27, 10, 2025);
7 // today.setDay(today.getDay() + 1); // COMPILER-TIME ERROR: l'oggetto è immutabile!
8 Date tomorrow = new Date(today.getDay() + 1, today.getMonth(), today.getYear());

```

Attenzione / Trappola d'Esame

Il Bug Persistente dei Getter e l'Effetto a Runtime: Anche in Lezione09, `Date.java` ha ancora il refuso di copia-incolla alle righe 43–44:

```

1 public int getMonth() { return day; } // Restituisce il campo day!
2 public int getYear() { return day; } // Restituisce il campo day!

```

Quando `MainDate` crea `tomorrow` invocando `today.getMonth()` su una data con `day = 27`, ottiene 27 come mese! Di conseguenza, il costruttore riceve (28, 27, 27) e `verify()` stampa: `Illegal date! tomorrow: 28/27/27` Nel tuo codice è opportuno correggere questi getter (`return month;` e `return year;`).

0.0.3 3. Ciclo: Corrispondenza con i Tuoi Moduli Workspace

Nel tuo repository [esercizi-wally-25-26](#): - Nel modulo `es09`: hai formalizzato questo esatto pattern creando `SingletonPattern.java` con costruttore privato e metodo factory `getInstance()`.

Nota di Studio

Con la **Lezione 09** abbiamo consolidato la manipolazione avanzata degli array, le proprietà degli enum, l'immutabilità e il pattern Singleton.

Il prossimo blocco è la **Lezione 10 / Slide T10: Java Variables and Call Stack**: - Modello formale della memoria JVM: **Call Stack (Stack Frames)** vs **Heap (Objects & Class Data)**. - Variabili d'istanza, variabili di classe (`static`), variabili locali e parametri di metodo. - Il passaggio dei parametri per valore (*pass-by-value*) per tipi primitivi vs reference types. - Il blocco di inizializzazione statica (`static { ... }`). - **Il grande spartiacque del corso**: il debutto del progetto **MavenDate**, con l'adozione ufficiale di Apache Maven, `pom.xml`, layout directory standard e primo packaging!

Dammi conferma per aprire la Lezione 10 / T10 e analizzare Call Stack e MavenDate!